Distributed ID Generation

Snowflake, UUID v7 vs UUID v4, ULID

Watch the 3-min video

In one line: generate IDs locally with no coordination but keep them time-ordered: a Snowflake-style 64-bit ID (timestamp + worker ID + sequence) or a UUID v7; random UUID v4 keys scatter B-tree inserts, and auto-increment needs one central database.

1 The problem: fifty instances, one ID space

Fifty instances of your order service take orders at the same time. Every order needs a **unique ID**, right now, **without asking a central database first**. And ideally, **newer orders get bigger IDs**, so IDs sort by time and inserts stay index-friendly.

Auto-increment is the classic answer: simple, compact (a 64-bit integer) and sorted by time. But every ID comes from **one database**, which becomes a bottleneck, and it doesn't work across shards (each shard would hand out the same numbers). It also **leaks information**: if your order number is 10,200, a competitor can place two orders a week apart and count your sales, or guess the next ID.

2 UUID v4: random, and why that hurts

UUID v4 is the opposite: **122 random bits** (plus 6 fixed version/variant bits), generated anywhere with no coordination. Collisions are practically impossible.

But as a primary key, randomness hurts. Each insert lands at a **random spot in the B-tree index**. That causes **page splits**, **poor cache locality** (the page you need is often not in memory) and a **bloated index** of half-full pages. At scale, inserts slow down noticeably. It is also 16 bytes, twice a BIGINT, in every index and foreign key.

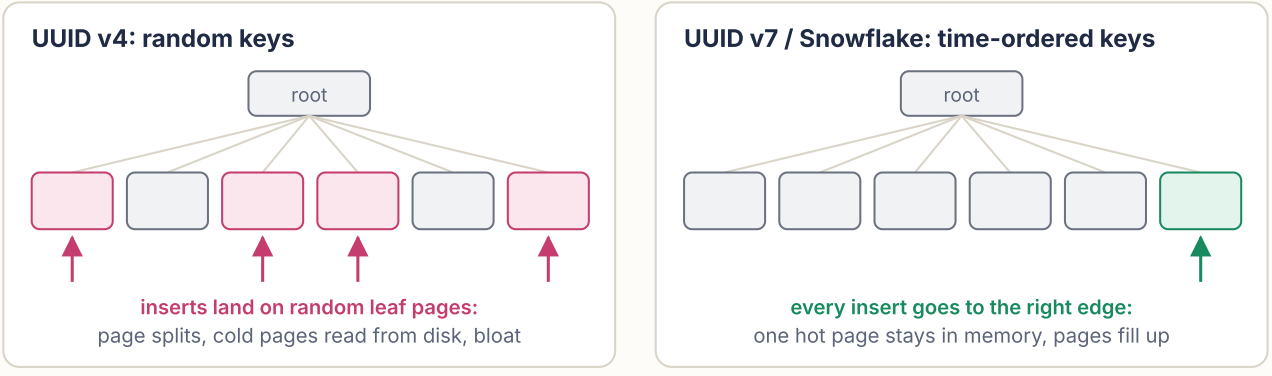


Figure 1. Random keys touch pages all over the index; time-ordered keys append at the right edge, like auto-increment.

3 Time-ordered IDs: UUID v7 and ULID

The fix is a **time-ordered ID**. **UUID v7**, standardized in 2024 (RFC 9562), starts with a **48-bit millisecond timestamp**, followed by random bits. **ULID** uses the same idea, encoded as a short, readable 26-character string. New IDs are always larger (to the millisecond), so inserts go to the **right edge of the index**, like auto-increment, but **without coordination**.

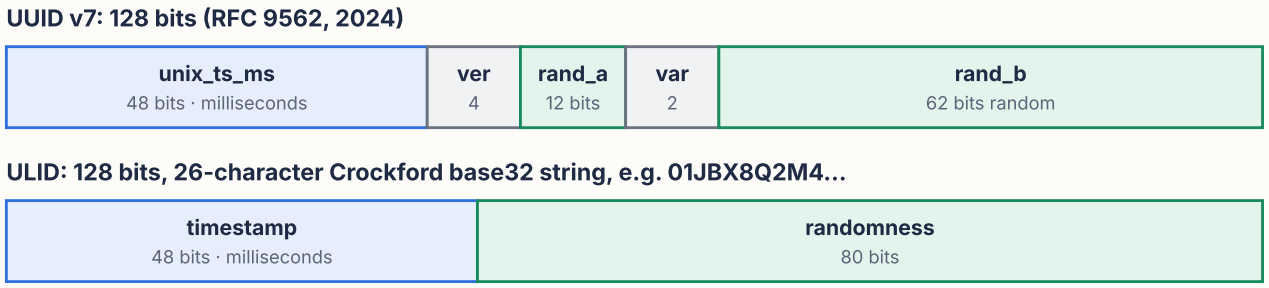


Figure 2. Both put the time first, so byte order equals time order. The random tail keeps IDs from different machines unique.

4 Snowflake: 64 bits, time-sorted

Twitter's **Snowflake** packs an ID into 64 bits, so it fits in a BIGINT:

- **41 bits of milliseconds** since a custom epoch: 2^{41} ms ≈ **69 years**.
- **10 bits of worker ID**: up to **1,024** generators.
- **12 bits of sequence**: each worker can create **4,096 IDs per millisecond** (about 4 million per second).

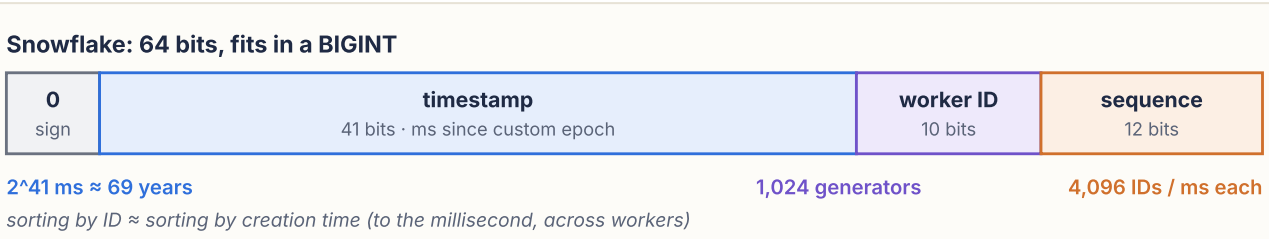


Figure 3. The top bit stays 0 so the ID is a positive signed 64-bit number.

In our order service, each of the 50 instances gets a **unique worker ID**, from configuration or a coordination service like **etcd** (lease a number, release it on shutdown). It builds IDs **locally, with no network calls**:

```
EPOCH = 1288834974657           # Twitter's epoch; pick your own

def next_id(self):
    now = current_ms()
    if now < self.last_ms:          # clock moved backwards
        raise ClockMovedBack()      # or wait until now >= last_ms
    if now == self.last_ms:
        self.seq = (self.seq + 1) & 0xFFF   # 12 bits
        if self.seq == 0:           # 4,096 used this ms
            now = wait_next_ms(self.last_ms)
    else:
        self.seq = 0
    self.last_ms = now
    return ((now - EPOCH) << 22) | (self.worker_id << 12) | self.seq
```

Watch out for clocks. If the system clock moves backwards (an NTP correction), the generator could repeat a timestamp and produce duplicates. It must **wait, or refuse**, until the clock passes the last timestamp it used. And two instances must never share a worker ID.

5 Other options, and public IDs

A **ticket server**, as Flickr used: a small database that only hands out auto-increment numbers. Often there are two, one giving **odd** and one giving **even** IDs, so either can fail. Simple, but it is still a network call per

ID (or per batch of IDs).

For **public URLs**, consider a separate, **opaque public ID** (a random string), so internal keys never leak. Note that time-ordered IDs also reveal **when** something was created.

Scheme	Size	Time-sorted	Coordination	Watch out for
Auto-increment	8 bytes	yes	one database	bottleneck, no shards, leaks counts
UUID v4	16 bytes	no	none	B-tree page splits, bloat
UUID v7 / ULID	16 bytes	yes (ms)	none	reveals creation time
Snowflake	8 bytes	yes (ms)	unique worker ID	clock going backwards
Ticket server	8 bytes	roughly	a call per ID or batch	extra hop, its availability

WORKED EXAMPLE: IS 12 BITS OF SEQUENCE ENOUGH?

- Peak traffic: **200,000 orders/s** across 50 instances, so about **4,000 orders/s per instance**, or 4 per millisecond.
- Each worker can make **4,096 per millisecond**: roughly 1,000× headroom. If a burst ever exceeds 4,096 in one ms, the generator just waits for the next millisecond.
- Worker IDs: 50 used of 1,024, room to scale out ~20×.
- Lifetime: 41 bits from a 2026 epoch last until about **2095**.

COMMON MISTAKES

- Random UUID v4 as the clustered primary key of a huge, insert-heavy table.
- Two instances with the same worker ID (e.g. copied config, or a restart reusing a lease too early): duplicates.
- Ignoring clock skew: a backwards NTP jump silently reissues IDs.
- Exposing sequential IDs in URLs, so anyone can count or enumerate your records.

6 Key terms

Auto-increment: database-assigned counter; central

UUID v4: 122 random bits; no order

UUID v7: 48-bit ms timestamp + random (RFC 9562)

ULID: 48-bit time + 80 random bits, 26 chars

Snowflake: 41 time + 10 worker + 12 sequence bits

Worker ID: unique number per generator

Page split: full B-tree page split in two on insert

Ticket server: central DB that only issues numbers

Opaque public ID: random external ID, hides internal keys

7 Interview questions

Q1. Design a distributed ID generator. BASIC

Snowflake-style 64-bit ID: 41-bit millisecond timestamp, 10-bit unique worker ID per instance, 12-bit per-millisecond sequence. Generated locally, sortable by time, 4,096 IDs/ms per worker. Handle clock skew by waiting or refusing. Or use UUID v7.

Q2. Why is UUID v4 bad for B-tree inserts? BASIC

Its randomness scatters inserts across the whole index: page splits, half-full pages (bloat) and poor caching, since the target page is often not in memory. Time-ordered keys append at the right edge instead.

Q3. What does the 41-bit timestamp in Snowflake give you? BASIC

2^41 milliseconds ≈ 69 years from the chosen epoch, and IDs that sort by creation time.

Q4. How do you assign worker IDs to 50 auto-scaled instances? SCENARIO

Lease one from a coordination service like etcd or ZooKeeper (a key with a TTL, renewed while alive), or derive it from a stable ordinal like a Kubernetes StatefulSet index. Never let two live instances hold the same ID.

Q5. NTP moves a server's clock back 200 ms. What should the generator do? SCENARIO

Never issue IDs for a timestamp it already used. Wait until the clock passes the last timestamp (fine for small jumps like 200 ms), or refuse and alert for large jumps.

Q6. You need a public order number that doesn't reveal sales volume. SCENARIO

Keep the internal Snowflake or UUID v7 key, and expose a separate random opaque ID (or a random UUID v4) in URLs and emails, with a unique index on it.

Q7. Are Snowflake IDs strictly ordered across all machines? TRICKY

No, only roughly. They are ordered within one worker; across workers, IDs created in the same millisecond order by worker ID, and clock skew between machines can reorder by a few milliseconds.

Q8. UUID v7 or Snowflake? TRICKY

UUID v7 needs no worker IDs at all and is a standard type, but costs 16 bytes. Snowflake fits in 8 bytes (smaller indexes and foreign keys) but needs unique worker ID assignment.

Next: SD-E-028 · Zero-Downtime Database Migrations backendroadmap.com